

Lekha Admin & Command Manual

Last updated: 2026-07-07

Bot version: matches `main` branch

How to keep this alive: when you add a new `/` command, new settings key, new admin flow, or new group behaviour, add a row/section here and bump the **Last updated** line. Treat this file as part of the shipped feature.

1. The one thing admins do most: let someone in

You do **not** need a user to paste their LINE ID. `/myid` replies with the ID as plain text, so they can select, copy, and paste it to an admin.

Approval options (pick any)

Method	How	Best for
Tap buttons in <code>/pending</code>	Type <code>/pending</code> → tap ✓ Allow on the user's card	Cleanest; no IDs needed
<code>/approve ``</code>	Approve by ID + auto-send welcome	They already sent you their ID
<code>/allow ``</code>	Add directly to allowlist (bypasses pending queue)	You sourced the ID yourself
<code>/promo <code></code> (user side)	User redeems <code>FREETRIAL100</code> → gets Team/allowed access automatically	Self-serve free trials

When you tap **Allow** on a pending card, the bot:

- moves them from `users:pending` → `users:allowed`
 - removes any trial flag
 - sends a welcome push if they weren't already allowed
 - logs `[admin] pending action via postback { admin, action, targetId }`
-

2. User-facing `/` slash commands

Command	Who	What it does
<code>/help</code>	anyone	Help card and command list (also works as <code>help</code>)
<code>/myid</code>	anyone	Sends your LINE user ID as plain text
<code>/settings</code>	allowed/trial/admin	Opens settings menu

Command	Who	What it does
<code>/settings <section></code>	allowed/trial/admin	Jumps to section: <code>briefing</code> , <code>tools</code> , <code>persona</code> , <code>memory</code> , <code>facts</code> , <code>locale</code>
<code>/settings:section:<section></code>	allowed/trial/admin	Postback-style section jump
<code>/settings:<section>:set:<key>:<value></code>	allowed/trial/admin	Direct setting change, e.g. <code>/settings:briefing:set:morningTime:07:30</code>
<code>/settings:toggle:<target>:off</code>	allowed/trial/admin	Disable <code>morning</code> , <code>evening</code> , or <code>checkin</code>
<code>/settings:facts:del:<id></code>	allowed/trial/admin	Delete a saved fact
<code>/set <key> <value></code>	allowed/trial/admin	Quick typed setting change
<code>/remember <fact></code>	allowed/trial/admin	Save a fact to memory
<code>/tutorial</code>	anyone	Restart onboarding tutorial
<code>/promo <code></code>	anyone	Redeem a promo code

`/set` keys and accepted values

Key	Aliases	Value	Example
<code>timezone</code>	<code>tz</code>	IANA timezone	<code>/set timezone Asia/Tokyo</code>
<code>location</code>	<code>loc</code>	free text	<code>/set location Bangkok</code>
<code>language</code>	<code>lang</code>	<code>en</code> , <code>th</code> , <code>auto</code>	<code>/set language th</code>
<code>morning</code>	—	HH:MM or off	<code>/set morning 07:30</code>
<code>evening</code>	—	HH:MM or off	<code>/set evening 21:30</code>
<code>checkin</code>	—	HH:MM or off	<code>/set checkin 18:00</code>
<code>compact</code>	<code>compactat</code> , <code>memorycompactat</code>	integer 1–1000	<code>/set compact 15</code>
<code>preferredname</code>	<code>personapreferredname</code>	free text	<code>/set preferredname Jim</code>

Settings sections and the keys inside them

`briefing`

- `morningBriefingTime` — HH:MM or null
- `eveningSummaryEnabled` — boolean
- `eveningSummaryTime` — HH:MM
- `taskCheckInEnabled` — boolean
- `taskCheckInTime` — HH:MM or derived from evening –30 min

- `inboxBriefingEnabled` — boolean
- `briefingTopics` — object of topic IDs → boolean (stocks, wellness, politics, crime, sports, business, entertain)
- `briefingLength` — "Headlines", "Bullets", "Full"
- `briefingLanguage` — "English", "ไทย", "EN + ไทย"
- `briefingChannels` — { line, email, push } booleans

tools

- `tools` — { todo, reminders, calendar, email, drive } booleans
- `disabledCategories` — derived array; do not edit directly

persona

- `personaTone` — "Warm", "Professional", "Playful"
- `personaAddressing` — "First name", "Khun", "Sir / Madam", "No address"
- `personaPrimaryLang` — "English", "Thai"
- `personaVoiceMatch` — boolean
- `personaPreferredName` — string or null

memory

- `memoryEnabled` — boolean
- `memoryCompactAt` — integer, messages between compactions

locale

- `timezone` — IANA string
- `location` — free text
- `language` — en, th, or null (auto)

facts

- Displays saved facts with **Delete** buttons.
- Facts have: `id` (short hex), `category`, `content`, `createdAt`, `updatedAt`, optional `confidence`, optional `priority`.
- Default category when added via `/remember` is `other`.
- Categories used internally: `preferences`, `people`, `habits`, `deadlines`, `context`, `health`, `work`, `other`.

3. Admin-only / slash commands

Command	Arguments	What it does
<code>/allow <LINE_USER_ID></code>	full LINE ID	Adds to allowlist, removes trial
<code>/remove <LINE_USER_ID></code>	full LINE ID	Removes from allowlist
<code>/users</code>	—	Lists every known user with status tags: <code>ADMIN</code> , <code>allowed</code> , <code>trial</code> , <code>team</code>
<code>/pending</code>	—	Shows pending queue as Flex cards with Allow / Deny buttons
<code>/approve <LINE_USER_ID></code>	full LINE ID	Approve from pending + send welcome
<code>/deny <LINE_USER_ID></code>	full LINE ID	Remove from pending queue
<code>/allowgroup <GROUP/ROOM_ID></code>	C... or R... ID	Allow a group/room
<code>/removegroup <GROUP/ROOM_ID></code>	C... or R... ID	Remove group/room from allowed list
<code>/groups</code>	—	Shows allowed + discovered groups with Allow / Remove buttons
<code>/promo create <code> [allowed team] [uses] [days]</code>	e.g. <code>/promo create DEMO team 10 7</code>	Creates a promo code
<code>/promos</code>	—	Lists all promo codes
<code>/promo delete <code></code>	code	Deletes a promo code
<code>/status <LINE_USER_ID></code>	full LINE ID	Full diagnostic: admin/allowed/trial/team, settings, locks, last activity
<code>/audit <LINE_USER_ID> [n]</code>	full LINE ID, optional count 1–100	Last <code>n</code> tool-call audit entries (default 5, max 100)
<code>/force-briefing <LINE_USER_ID> [morning evening]</code>	full LINE ID, kind	Manually push a briefing/summary

Why `/users` now shows more than just "allowed"

A user can be in several states at once:

- **ADMIN** — in `ADMIN_LINE_USER_ID` env var; bypasses all gates.
- **allowed** — in `users:allowed` Redis set.
- **trial** — in `users:trial` (e.g. started a free trial but hasn't redeemed a promo).
- **team** — in `users:team` (e.g. redeemed `FREETRIAL100` or paid Team plan).

So you will now see lines like:

```
James (U9b7...) [ADMIN]
Keno (U4e1...) [allowed]
dang (Ue00...) [trial]
SKY~💕 (U898...) [trial]
```

```
Pol (U7c4...) [ADMIN]
pang (Ub81...) [allowed]
```

This is why Keno and Pang show `allowed` — they explicitly used `/promo FREETRIAL100`, which writes them into `users:allowed` (or `users:team` depending on the code grant). James and Pol are `ADMIN`. Dang and Sky are on free trial (`trial`).

4. Group management

How the bot knows about groups

- When the bot is **added to any group or room**, it stores the ID in `groups:discovered`.
- If the inviter is an **admin**, the group is automatically added to `groups:allowed`.
- If the inviter is **not an admin**, the group sees the Team paywall, and every admin gets a push notification with **Allow / Ignore** buttons.
- When the bot leaves or is removed, the group is removed from `groups:allowed` and `groups:discovered`.

How to allow a group for free

Method	Steps
Tap the admin notification	When a non-admin adds the bot, admins get a Flex card. Tap ✓ Allow .
<code>/groups</code>	Type <code>/groups</code> → see discovered groups → tap ✓ Allow .
<code>/allowgroup ``</code>	Type the exact group/room ID.
Add to <code>ADMIN_GROUP_IDS</code>	Vercel env var; permanent, survives restarts.

How users talk to the bot in a group

LINE does **not** show bots in the group `@` autocomplete, so members cannot pick Lekha from the mention picker. They can still invoke the bot with:

1. **@Lekha typed manually** at the start of a message (case-insensitive).
2. **The name at the start** — e.g. `Lekha, what's the weather?`.
3. **A reply to any of the bot's messages** — swipe/right-click the bot's message and hit reply; the bot treats that as a direct invocation, no `@` needed.

Tip: the bot's join message already tells members they can mention `@Lekha` or reply to its messages.

`/groups` output

- **✓ Allowed groups** — already authorised (`groups:allowed` + `ADMIN_GROUP_IDS`).

- 🕒 **Discovered, not allowed** — bot is in the group but it cannot respond yet. Tap **Allow** to activate.
 - Each card shows the full group/room ID and a short prefix, plus an action button.
-

5. Promo codes

Promo codes live in Redis at `promo:<CODE>` .

Field	Meaning
<code>grant</code>	"allowed" (personal access) or "team" (Team access)
<code>usesLeft</code>	how many redemptions remain
<code>expiresAt</code>	epoch ms expiration
<code>createdBy</code>	admin LINE ID
<code>usedBy</code>	set of user IDs who already redeemed

`/promo FREETRIAL100` is the current self-serve code. When a user redeems, the bot logs:

```
[promo] redeem attempt { userId, code, ok, grant, error }
```

6. Audit & status

`/audit <id> [n]`

- Shows the last `n` turns for a user (default 5, max **100**).
- Each entry includes: timestamp, hint, user message, every tool call (input/output), errors, and the final reply.
- Use this to trace exactly what the bot did and why.

`/status <id>`

- Admin / allowed / trial / team flags
 - Timezone, morning/evening/check-in times
 - Whether LINE pushes are enabled
 - Last briefing/summary timestamps
 - Today's push locks
 - Last activity relative time
-

7. Push vs reply: when the bot uses each

Scenario	Method	Counts against push quota?
Answering a message you just sent	reply (uses replyToken)	No
Proactive briefings, reminders, alerts	push	Yes
Admin notifications for new groups	push	Yes
Welcome message after <code>/approve</code>	push	Yes
<code>/myid</code>	reply	No
<code>/pending</code> , <code>/users</code> , <code>/groups</code>	reply	No

We prefer replies whenever there is a reply token, to save push quota.

8. Fact storage deep reference

Facts are stored per user at `user:{userId}:facts:v2`.

Fact shape

```
{
  id: string;           // short hex, e.g. "a1b2c3d4e5f6"
  category: "preferences" | "people" | "habits" | "deadlines" | "context" | "health" | "wo
  content: string;      // max 1000 chars
  createdAt: number;
  updatedAt: number;
  confidence?: number;  // optional 0-1
  priority?: number;    // optional; higher = shown first in prompt
}
```

How facts are added

- `/remember <fact>` → category `other`
- Background extraction after every `memoryCompactAt` turns → categorized automatically
- Document / image analysis can create facts with higher `priority`

How facts are removed

- Settings → **Facts** section → tap **Delete** on a fact card (`/settings:facts:del:<id>`)
- There is no bulk-delete command by design

9. Changelog / future additions

Use this section as a running log. Add a bullet every time the manual is updated.

- **2026-07-07** — Converted `=` commands to `/` commands; added `/myid` plain-text reply; added group discovery and admin `/groups` Flex cards; increased `/audit` max to 100; improved `/users` status logging; created this manual.
-

10. Quick admin cheat sheet

```
/users          → see everyone and their tags
/pending        → approve/deny with buttons
/allow <id>     → add someone directly
/approve <id>   → approve from pending + welcome
/groups         → see/allow discovered groups
/allowgroup <id> → allow a group by ID
/promos        → list promo codes
/promo create X team 50 30
/status <id>    → full user diagnostic
/audit <id> 20  → last 20 turns
/force-briefing <id> morning
```